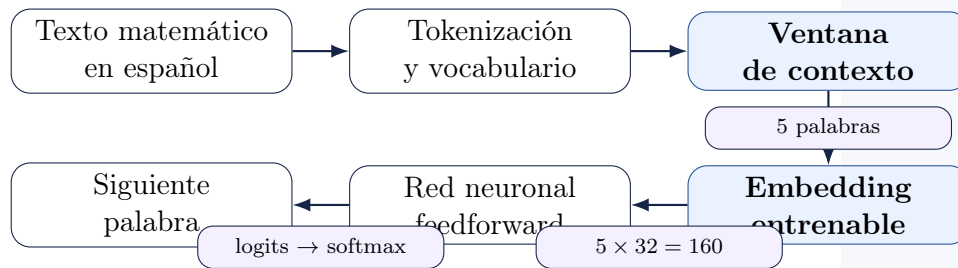


Tiny Language Model

Modelo neuronal para predicción de la siguiente palabra

Memoria del proyecto

Proyecto de aprendizaje automático aplicado a texto matemático en español



Idea central: convertir palabras en vectores, usar una ventana fija de contexto y entrenar una red neuronal para predecir la palabra posterior.

Autor	Mario Gabarrón Espín
Proyecto	Tiny Language Model
Lenguaje	Python
Librería principal	PyTorch
Interfaz	Tkinter
Tarea	Predicción de siguiente palabra

Esta memoria continúa la línea del proyecto anterior de red neuronal desde cero (**el otro proyecto subido en mi GitHub**), pero trasladando la idea de aprendizaje neuronal desde imágenes y píxeles hacia texto, vocabulario, contexto y embeddings.

Índice

1. Introducción	3
2. Objetivos del proyecto	3
3. Del reconocimiento de dígitos al lenguaje	4
4. Tokenización y vocabulario	4
4.1. Tokenización	4
4.2. Vocabulario	5
5. El concepto de contexto	5
6. Representación de palabras: one-hot frente a embedding	6
6.1. Representación one-hot	6
6.2. Embedding	6
7. La tabla de embeddings	6
8. Diferencia entre vocab_size, context_size y embedding_dim	7
9. Arquitectura del modelo	8
10. Propagación hacia delante	8
11. Softmax y predicción	9
12. Función de coste	9
13. Retropropagación y optimizador	10
14. Batch size	10
15. Tasa de aciertos: accuracy	10
16. Por qué la accuracy puede estabilizarse	11

17.Top-k accuracy como mejora futura	12
18.Interfaz gráfica	12
19.Limitaciones del modelo	12
20.Posibles mejoras	13
21.Conclusión	13
A. Código fuente de <code>tlm_class.py</code>	14

1. Introducción

Este proyecto consiste en la construcción de un pequeño modelo de lenguaje, llamado **Tiny Language Model** o **TLM**, cuyo objetivo es predecir la siguiente palabra a partir de un contexto previo.

A diferencia del proyecto anterior de red neuronal desde cero (**el otro proyecto subido en mi GitHub**), en el que se construía una red neuronal para reconocer dígitos dibujados en una cuadrícula de píxeles, en este caso el objetivo no es clasificar una imagen, sino trabajar con texto. La entrada ya no es una matriz de píxeles, sino una secuencia de palabras.

El modelo se ha construido con **PyTorch**, manteniendo una estructura lo más cercana posible a la red neuronal anterior (**el otro proyecto subido en mi GitHub**):

entrada $\longrightarrow$ capa oculta $\longrightarrow$ capa oculta $\longrightarrow$ salida

Sin embargo, aparece un nuevo concepto fundamental: el **embedding**. Este permite transformar palabras en vectores numéricos densos que pueden ser procesados por una red neuronal.

Idea clave

El objetivo del modelo no es conversar como un modelo de lenguaje grande, sino aprender el mecanismo básico de predicción de la siguiente palabra: dado un contexto de varias palabras, el modelo calcula cuál es la palabra más probable a continuación.

2. Objetivos del proyecto

Los objetivos principales del proyecto son:

- Construir un pequeño modelo de lenguaje en Python usando PyTorch.
- Comprender cómo se representa el texto de forma numérica.
- Introducir el concepto de **contexto** como ventana de palabras anteriores.
- Comprender el papel de los **embeddings** en modelos de lenguaje.
- Entrenar una red neuronal para predecir la siguiente palabra.
- Implementar una función de coste mediante **CrossEntropyLoss**.
- Aplicar retropropagación usando **backward()** y un optimizador.
- Medir el rendimiento mediante una tasa de aciertos o **accuracy**.
- Diseñar una interfaz gráfica para entrenar y probar el modelo.

3. Del reconocimiento de dígitos al lenguaje

En el proyecto anterior de red neuronal desde cero (**el otro proyecto subido en mi GitHub**), la red neuronal recibía como entrada una imagen representada por píxeles. Por ejemplo, una cuadrícula de 25×25 píxeles se convertía en un vector de 625 valores:

$$25 \cdot 25 = 625$$

Cada neurona de entrada representaba un píxel.

En este proyecto, la entrada ya no es una imagen, sino texto. Por tanto, la pregunta fundamental cambia:

Proyecto	Entrada	Salida
Red de dígitos	Píxeles de una imagen	Dígito predicho
Tiny Language Model	Palabras de contexto	Siguiente palabra

Idea clave

La arquitectura neuronal sigue siendo parecida, pero cambia la representación de la entrada. En lugar de píxeles, ahora se procesan palabras. Para que una red neuronal pueda trabajar con palabras, primero hay que convertirlas en números.

4. Tokenización y vocabulario

4.1. Tokenización

La tokenización consiste en convertir un texto en unidades más pequeñas, llamadas **tokens**. En este proyecto, los tokens son simplemente palabras.

Por ejemplo, la frase:

`la derivada mide el cambio instantaneo`

se transforma en la lista:

`[la, derivada, mide, el, cambio, instantaneo]`

Antes de tokenizar, el texto se normaliza: se convierte a minúsculas, se eliminan tildes, se eliminan signos extraños y se separa en palabras.

4.2. Vocabulario

El vocabulario es el conjunto de palabras distintas que conoce el modelo. Cada palabra se asocia a un índice numérico.

Palabra	Índice
<pad>	0
<unk>	1
<eos>	2
derivada	50
funcion	49
cambio	191

Los tokens especiales son <pad> para relleno, <unk> para palabras desconocidas y <eos> para final de frase.

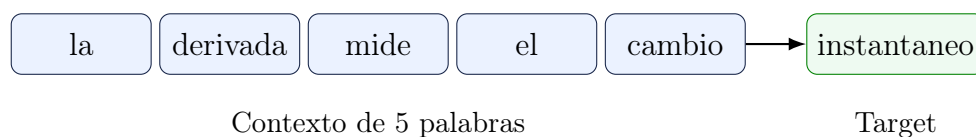
5. El concepto de contexto

El **contexto** es el conjunto de palabras anteriores que el modelo utiliza para predecir la siguiente palabra. Si se toma:

`context_size = 5`

entonces el modelo mira las últimas cinco palabras.

[la, derivada, mide, el, cambio] $\longrightarrow$ instantaneo



Idea de ventana deslizante

A partir de una frase larga se generan muchos ejemplos de entrenamiento desplazando una ventana de tamaño fijo. Cada ventana se usa como entrada, y la palabra inmediatamente posterior se usa como objetivo.

6. Representación de palabras: one-hot frente a embedding

6.1. Representación one-hot

Una primera forma de representar palabras sería usar vectores **one-hot**. Si el vocabulario tiene V palabras, cada palabra se representa con un vector de tamaño V . Por ejemplo, si $V = 500$, cada palabra se representa mediante un vector de 500 componentes, donde sólo una vale 1 y las demás valen 0.

Esto tiene dos problemas: los vectores son muy grandes y no expresan relación entre palabras. Por ejemplo, las palabras **derivada** e **integral** serían igual de distintas que **derivada** y **mesa**, porque todos los vectores one-hot son ortogonales entre sí.

6.2. Embedding

El **embedding** resuelve este problema asignando a cada palabra un vector denso de tamaño menor. Si:

$$\text{embedding_dim} = 32$$

entonces cada palabra se representa mediante 32 números reales:

$$\text{palabra} \longrightarrow \text{vector de 32 componentes}$$

Idea clave

El embedding no se fija manualmente. Es una matriz entrenable. Sus valores se ajustan durante el entrenamiento igual que los pesos de las capas lineales.

7. La tabla de embeddings

En PyTorch, la capa:

```
nn.Embedding(vocab_size, embedding_dim)
```

crea una matriz entrenable:

$$E \in \mathbb{R}^{V \times d}$$

donde V es el tamaño del vocabulario y d es la dimensión del embedding. Cada fila de la matriz representa una palabra del vocabulario.

		Columnas: <code>embedding_dim</code>			
Filas: vocabulario	<pad>	e_1	e_2	e_3	e_4
	derivada	e_1	e_2	e_3	e_4
	funcion	e_1	e_2	e_3	e_4
	cambio	e_1	e_2	e_3	e_4

Si el vocabulario tiene 345 palabras y la dimensión del embedding es 32, entonces:

$$E \in \mathbb{R}^{345 \times 32}$$

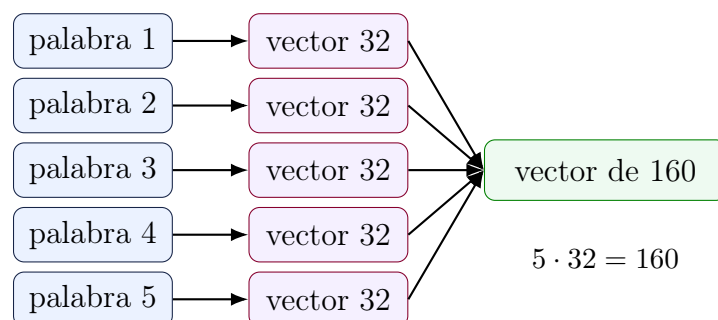
8. Diferencia entre `vocab_size`, `context_size` y `embedding_dim`

Concepto	Significado	Ejemplo
<code>vocab_size</code>	número de palabras conocidas	345
<code>context_size</code>	palabras anteriores que mira el modelo	5
<code>embedding_dim</code>	números usados para representar cada palabra	32

Si `context_size` = 5 y `embedding_dim` = 32, entonces la entrada real a la primera capa lineal tiene tamaño:

$$5 \cdot 32 = 160$$

No se usa `vocab_size` · `embedding_dim` porque no se introducen todas las palabras del vocabulario a la vez. Sólo se seleccionan las palabras del contexto.



9. Arquitectura del modelo

La arquitectura implementada es:

índices de palabras $\rightarrow$ embedding $\rightarrow$ flatten $\rightarrow$ 120 $\rightarrow$ 60 $\rightarrow$ vocab_size

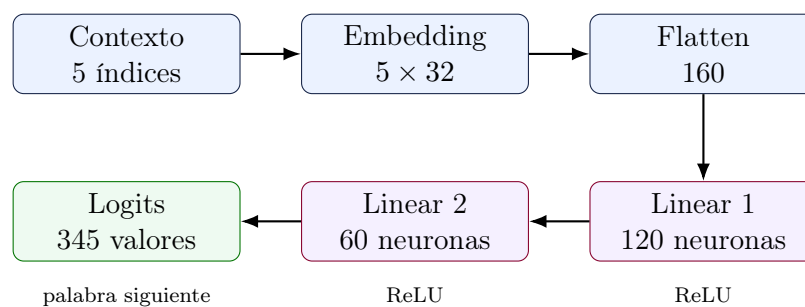
Con los valores del entrenamiento de referencia:

vocab_size = 345, context_size = 5, embedding_dim = 32

la arquitectura efectiva es:

5 palabras $\rightarrow$ $5 \times 32 \rightarrow 160 \rightarrow 120 \rightarrow 60 \rightarrow 345$

El siguiente esquema se ha dispuesto en dos líneas para que no se salga de los márgenes de la página:



10. Propagación hacia delante

El método `forward` define cómo fluye la información a través de la red. La entrada es un tensor de índices con forma:

`(batch_size, context_size)`

Primero, la capa embedding convierte cada índice en un vector denso:

`embedded = embedding(x)`

Después, el tensor se aplanar:

`(batch_size, context_size, embedding_dim)  $\rightarrow$  (batch_size, context_size*embedding_dim)`

Finalmente se aplican las capas lineales:

$$z_1 = W_1x + b_1, \quad a_1 = \text{ReLU}(z_1)$$

$$z_2 = W_2a_1 + b_2, \quad a_2 = \text{ReLU}(z_2)$$

$$z_3 = W_3a_2 + b_3$$

El resultado z_3 recibe el nombre de **logits**. Estos logits no son probabilidades todavía.

Idea clave

Durante el entrenamiento no se aplica **softmax** dentro del método **forward**, porque **CrossEntropyLoss** trabaja directamente con logits.

11. Softmax y predicción

Para convertir los logits en probabilidades se utiliza la función **softmax**:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

La predicción final se obtiene tomando el índice con mayor probabilidad:

$$\text{prediccion} = \arg \max_i p_i$$

12. Función de coste

La función de coste utilizada es **CrossEntropyLoss**. Esta función recibe los logits producidos por la red y el índice de la palabra correcta. El target no se representa como one-hot, sino como un índice.

Interpretación

Si el modelo asigna alta probabilidad a la palabra correcta, el coste es bajo. Si asigna baja probabilidad a la palabra correcta, el coste es alto.

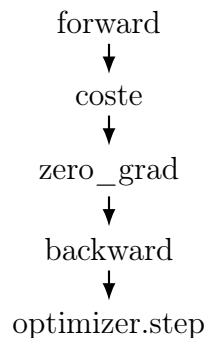
13. Retropropagación y optimizador

En el proyecto anterior de red neuronal desde cero (**el otro proyecto subido en mi GitHub**), la retropropagación se calculaba manualmente mediante derivadas, deltas y actualización explícita de pesos. En este proyecto, PyTorch calcula automáticamente los gradientes mediante:

```
cost.backward()
```

El optimizador se encarga de actualizar los parámetros de la red. En este proyecto se utiliza Adam:

```
torch.optim.Adam
```



14. Batch size

El `batch_size` indica cuántos ejemplos se entrenan simultáneamente en una iteración. Si `batch_size=32`, entonces la red procesa 32 contextos al mismo tiempo.

$$x.shape = (32, 5)$$

significa que se procesan 32 ejemplos y que cada ejemplo tiene 5 palabras de contexto.

15. Tasa de aciertos: accuracy

La accuracy mide el porcentaje de ejemplos en los que la palabra más probable coincide exactamente con el target.

$$\text{accuracy} = \frac{\text{número de aciertos}}{\text{número total de ejemplos}} \cdot 100$$

Un ejemplo se considera correcto si:

$$\arg \max(\text{logits}) = \text{target}$$

Contexto	Target	Predicción
la derivada mide el cambio	instantaneo	instantaneo
una matriz es una tabla	rectangular	rectangular
en matematicas el concepto de	suma	derivada

Resultado del entrenamiento

En un entrenamiento de referencia se obtuvo una accuracy aproximada de 79,86 % con:

`vocab_size = 345, context_size = 5, embedding_dim = 32`

y 100 épocas de entrenamiento.

16. Por qué la accuracy puede estabilizarse

En un modelo de predicción de palabras, la accuracy exacta puede estabilizarse aunque el modelo esté aprendiendo correctamente. Esto ocurre porque existen contextos iguales con targets distintos.

Por ejemplo:

`en matematicas el concepto de`

puede ir seguido de:

`suma, resta, derivada, matriz`

Por tanto, el mismo input puede tener varias salidas posibles. La red no puede acertarlas todas si sólo se permite una predicción.

Ambigüedad del lenguaje

A diferencia de una imagen de un dígito, una secuencia de palabras puede tener muchas continuaciones válidas. Por eso la accuracy exacta no siempre refleja toda la calidad del modelo.

17. Top-k accuracy como mejora futura

Una mejora natural consiste en implementar **top-k accuracy**. La top-k accuracy no exige que la palabra correcta sea la primera predicción, sino que aparezca entre las k palabras más probables.

Por ejemplo, con $k = 5$, el modelo acierta si el target aparece entre sus cinco predicciones principales.

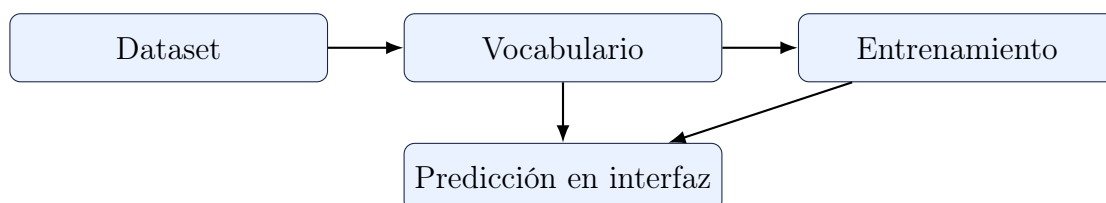
Top 5 predicciones	Target
derivada, funcion, suma, matriz, integral	suma

Mejora futura

La top-k accuracy es especialmente adecuada para modelos de lenguaje, porque muchas veces existen varias palabras razonables para continuar una frase.

18. Interfaz gráfica

El proyecto incluye una interfaz gráfica en Tkinter que permite seleccionar datasets, configurar hiperparámetros, entrenar el modelo, visualizar la barra de progreso, mostrar coste y accuracy, guardar el modelo entrenado, cargar un modelo guardado, escribir un texto y visualizar la predicción de la siguiente palabra.



19. Limitaciones del modelo

El modelo desarrollado es intencionadamente pequeño y educativo. Sus principales limitaciones son:

- sólo predice la siguiente palabra;
- usa una ventana fija de contexto;
- no utiliza mecanismos de atención;
- el dataset es pequeño y temático;

- algunas frases tienen estructuras repetidas que generan ambigüedad;
- el modelo no comprende matemáticas en sentido profundo, sino patrones estadísticos de palabras.

20. Posibles mejoras

Algunas mejoras futuras serían implementar top-k accuracy, aumentar el tamaño del dataset, usar frases menos repetitivas, aumentar el `context_size`, generar texto de varias palabras de forma autoregresiva, añadir temperatura, separar entrenamiento y validación, usar embeddings más grandes y experimentar con redes recurrentes o Transformers.

21. Conclusión

Este proyecto representa una evolución natural respecto al proyecto anterior de red neuronal desde cero (**el otro proyecto subido en mi GitHub**). En lugar de clasificar imágenes, se trabaja con lenguaje. Para ello se introducen nuevos conceptos fundamentales: tokenización, vocabulario, contexto, embeddings, logits, softmax, entropía cruzada y accuracy.

Idea clave

El valor principal del proyecto no está en crear un modelo perfecto, sino en comprender cómo una red neuronal puede pasar de trabajar con píxeles a trabajar con palabras.

A. Código fuente de `tlm_class.py`

A continuación se incluye el código principal del modelo neuronal utilizado en el proyecto. Se ha formateado para que los comentarios no rompan visualmente las líneas del documento.

```

1  import torch
2  import torch.nn as nn
3
4
5  # batch_size determines how many samples are used
6  # to train the neural network in one iteration.
7
8  class TLM(nn.Module):
9      """
10         Tiny Language Model.
11
12         Architecture:
13
14             vocab_size -> 120 -> 60 -> vocab_size
15
16         The input and output dimensions depend on vocab_size.
17         The hidden layers have 120 and 60 neurons.
18         """
19
20     def __init__(self, vocab_size, context_size, embedding_dim):
21         """
22         Initializes the neural network.
23
24         vocab_size:
25             number of unique words in the vocabulary.
26
27         context_size:
28             number of previous words used to predict the next word.
29
30         embedding_dim:
31             size of the dense vector that represents each word.
32         """
33
34         super().__init__()
35
36         self.vocab_size = vocab_size
37         self.context_size = context_size
38         self.embedding_dim = embedding_dim
39
40         # Each context has context_size words.
41         # Each word is represented with embedding_dim numbers.
42         input_size = embedding_dim * context_size
43
44         # Layers of the neural network.
45         self.layer1 = nn.Linear(input_size, 120)
46         self.layer2 = nn.Linear(120, 60)
47         self.layer3 = nn.Linear(60, vocab_size)
48
49         # Activation and loss function.
50         self.relu = nn.ReLU()
51         self.lossFunction = nn.CrossEntropyLoss()
52
53         # Embedding table.
54         self.embedding = nn.Embedding(vocab_size, embedding_dim)
55
56     def forward(self, x):
57         """
58         Input:
59             x with shape (batch_size, context_size)
60
61         Output:
62             logits with shape (batch_size, vocab_size)
63         """
64

```

```
65         embedded = self.embedding(x)
66
67         # Shape:
68         # (batch_size, context_size, embedding_dim)
69
70         embedded = embedded.flatten(start_dim=1)
71
72         # Shape:
73         # (batch_size, context_size * embedding_dim)
74
75         z1 = self.layer1(embedded)
76         a1 = self.relu(z1)
77
78         z2 = self.layer2(a1)
79         a2 = self.relu(z2)
80
81         # Final logits.
82         z3 = self.layer3(a2)
83
84         return z3
85
86     def prediction(self, input):
87         """
88         Predicts the next word index.
89         """
90
91         with torch.no_grad():
92             logits = self.forward(input)
93             probabilities = torch.softmax(logits, dim=-1)
94             prediction = torch.argmax(probabilities, dim=-1)
95
96         return prediction
97
98     def cost(self, logits, target):
99         """
100         Computes the loss from logits and target indices.
101         Softmax is not applied before CrossEntropyLoss.
102         """
103
104         return self.lossFunction(logits, target)
105
106     def backpropagation(self, cost, optimizer):
107         """
108         Applies backpropagation and gradient descent.
109         """
110
111         # PyTorch accumulates gradients, so they must be reset.
112         optimizer.zero_grad()
113
114         # Computes gradients.
115         cost.backward()
116
117         # Updates weights and biases.
118         optimizer.step()
119
120     def trainStep(self, input, target, epochs, optimizer):
121         """
122         Trains the neural network for a given number of epochs.
123         """
124
125         total_cost = 0
126
127         for e in range(epochs):
128             logits = self(input)
129             cost = self.cost(logits, target)
130             self.backpropagation(cost, optimizer)
131             total_cost += cost.item()
132
133         return total_cost / epochs
134
135     def accuracy(self, input, target, batch_size=32):
136         """
```

```
137         Calculates the accuracy of the neural network.
138
139         input:
140             tensor with shape (number_of_examples, context_size)
141
142         target:
143             tensor with shape (number_of_examples)
144
145         output:
146             accuracy percentage.
147         """
148
149         was_training = self.training
150         self.eval()
151
152         total_predictions = 0
153         correct_predictions = 0
154
155         with torch.no_grad():
156             for start in range(0, input.shape[0], batch_size):
157                 batch_input = input[start:start + batch_size]
158                 batch_target = target[start:start + batch_size]
159
160                 logits = self(batch_input)
161                 predictions = torch.argmax(logits, dim=-1)
162
163                 correct_predictions += (
164                     predictions == batch_target
165                 ).sum().item()
166
167                 total_predictions += batch_target.shape[0]
168
169         if was_training:
170             self.train()
171
172         accuracy = correct_predictions / total_predictions
173
174         return accuracy * 100
```

Listing 1: Clase principal TLM